Truth Code Standards

Document: `code-standards.md`
Project: Truth
Purpose: Define mandatory standards for code quality, architecture, safety, testing, and maintainability.

---

1. Core Principles

All code contributed to Truth MUST follow these principles:

- **Clarity over cleverness:** Write code that another developer can understand without reverse engineering it.
- **Small, controlled changes:** Implement one coherent unit of work at a time.
- **Explicit responsibilities:** Each module, class, and function must have a clear purpose.
- **Predictable behavior:** Prefer deterministic, testable behavior over hidden side effects.
- **Preserve existing functionality:** Do not break working behavior while implementing unrelated features.
- **Security by default:** Validate inputs, restrict permissions, and protect user data.
- **Documented uncertainty:** Never present assumptions, experimental interpretations, or simulated results as established facts.
- **Incremental evolution:** Design for future expansion without implementing speculative infrastructure prematurely.

---

2. Repository and File Organization

- Follow the repository's existing structure unless an approved task requires a structural change.
- Before creating, moving, renaming, or deleting files:
    - Inspect the relevant directories and existing implementation.
    - Check for an existing module that already performs the required function.
    - Reuse established patterns where appropriate.
    - Update affected imports, tests, documentation, and configuration when necessary.
- Avoid duplicate modules that implement the same responsibility.
- Do not reorganize the entire repository merely to satisfy personal stylistic preferences.

- Keep configuration, application logic, tests, documentation, and generated artifacts separated according to the established project structure.
- Do not commit secrets, temporary debugging files, local databases containing private data, build outputs, or unrelated generated files.

---

3. Naming Conventions

Use descriptive, consistent names:

- **Python:**
  - Modules and packages: `snake_case`
  - Functions and variables: `snake_case`
  - Classes: `PascalCase`
  - Constants: `UPPER_SNAKE_CASE`
  - Private implementation details: leading underscore where appropriate (`_private_detail`)
  - Boolean variables: names that express a condition, such as `is_valid`, `has_permission`, or `can_execute`.

Avoid unexplained abbreviations, ambiguous names, and generic identifiers such as `data`, `thing`, or `manager` when a more precise name is available. Use terminology consistently across the Truth codebase. Do not casually rename established concepts such as Kernel, State, Memory, Reasoning, Observation, Response, or Commit.

---

4. Functions and Classes

- Every function or class MUST have a clearly defined responsibility.
- Keep functions focused on one logical operation.
- Use parameters and return values to make dependencies explicit.
- Avoid excessive nesting and unnecessarily complicated conditional logic.
- Extract repeated logic into reusable functions when doing so improves clarity.
- Avoid creating abstractions for hypothetical future requirements.
- Avoid classes that merely wrap functions without adding meaningful state or behavior.
- Prefer composition when it makes dependencies and responsibilities clearer.
- Use type annotations for public functions, important data structures, and module boundaries.
- Add docstrings to public modules, classes, and functions when their purpose or behavior is not self-evident.
- Do not introduce global mutable state unless an explicitly approved design requires it.

- Avoid hidden network requests, file writes, database changes, or other side effects inside functions that appear to perform simple calculations.

---

5. Truth Kernel Boundaries

The Kernel is the coordinating foundation of Truth. Its boundaries MUST remain explicit. Maintain separation between the following responsibilities:

1. **Observation:** Receives and normalizes incoming information.
2. **Memory Retrieval:** Retrieves relevant stored information through defined interfaces.
3. **State:** Represents the current known state and relevant context.
4. **Reasoning:** Evaluates inputs and state to produce conclusions or proposed decisions.
5. **Response:** Formats and returns the result to the appropriate interface.
6. **Commit:** Records approved state changes, events, or other permitted outcomes.

Established execution flow:
`Input` → `Observation` → `Memory Retrieval` → `State` → `Reasoning` → `Response` → `Commit`

Boundary Rules

- Do not merge all Kernel responsibilities into one oversized function or module.
- Do not let interfaces bypass the Kernel's approved execution and permission boundaries.
- Do not let reasoning components silently write to persistent memory.
- Do not allow memory retrieval to modify unrelated application state.
- Do not let response formatting become the source of truth for application state.
- Keep persistence behind explicit interfaces when the existing architecture supports it.
- Make state changes identifiable, reviewable, and testable.
- Preserve the ability to replace the current deterministic reasoning implementation with an AI-backed implementation without rewriting the entire Kernel.

---

6. State, Memory, and Data Integrity

Truth must distinguish between information it receives, information it stores, conclusions it derives, and actions it executes:

- **Observation:** Information received from a source.

- **Stored memory:** Information retained according to an approved persistence policy.
- **Inference:** A conclusion derived from available information.
- **Uncertainty:** Information that is missing, ambiguous, disputed, or insufficiently supported.
- **State transition:** A defined change from one application state to another.
- **Action:** An operation that may produce an external or persistent effect.
- Do not silently convert an inference into a verified fact.
- Preserve source references, timestamps, confidence indicators, or provenance metadata when required by the relevant data model. Do not invent confidence values or provenance information.
- Validate data at system boundaries using explicit schemas or typed models.
- Handle missing values, invalid formats, conflicting observations, and unexpected state transitions deliberately.
- Never modify historical records or user memories in place when the system requires an auditable history. Use an approved correction, versioning, or event-recording strategy.

---

7. Error Handling and Logging

- Handle errors explicitly and predictably.
- Catch exceptions only when the code can meaningfully recover, translate, or handle them. Do not silently swallow exceptions or use broad exception handling as a substitute for understanding failure conditions.
- Return structured errors at defined system boundaries.
- Keep internal diagnostic details separate from information exposed to users.
- Use appropriate logging levels. Avoid logging passwords, API keys, authentication tokens, private journals, or unnecessary personal information.
- Never rely on console output as the application's permanent error-reporting mechanism.
- Failures in memory retrieval, reasoning, persistence, or external integrations must not be misrepresented as successful operations. If a commit fails, do not report a state change as permanently saved unless the persistence layer confirms success.

---

8. Security and Permissions

- Validate and constrain untrusted inputs.
- Use parameterized database operations and safe serialization methods.
- Never execute user-supplied text as code without an explicitly designed and secured execution environment.
- Do not hardcode credentials or secrets. Use environment variables or an approved secrets-management mechanism.
- Apply least-privilege permissions and protect sensitive logs/stored information.
- Avoid exposing internal file paths, stack traces, credentials, or implementation details through public interfaces.
- Treat external content, retrieved documents, and model-generated instructions as untrusted data.
- Actions with meaningful external consequences require appropriate authorization, validation, and confirmation policy. Never bypass safeguards to make an agent appear more autonomous.

---

9. Configuration and Dependencies

- Keep environment-specific configuration outside application logic.
- Do not hardcode machine-specific paths, hostnames, credentials, or deployment assumptions.
- Validate required configuration during startup or at the relevant boundary.
- Pin or constrain dependencies according to the repository's existing dependency-management strategy.
- Avoid adding a dependency for functionality that can be implemented clearly and safely with existing tools.
- Do not introduce a new database, message broker, AI provider, framework, or external service without a clear requirement and an approved architectural decision.

---

10. Testing Standards

Every meaningful change MUST have an appropriate verification strategy using applicable test categories:

- **Unit tests:** Test individual functions, classes, and transformations.
- **Integration tests:** Test communication between Kernel stages and external boundaries.

- **Regression tests:** Reproduce previously discovered bugs and confirm existing behavior remains intact.
- **Security tests:** Verify input validation, permission enforcement, and failure isolation.

Tests MUST NOT depend on live external services unless explicitly designed as integration or end-to-end tests. Do not weaken or delete existing tests simply because the implementation fails them.

---

11. API and Interface Contracts

- Define accepted inputs and returned outputs for API endpoints and service boundaries.
- Validate request data, use appropriate HTTP status codes, and return structured error responses.
- Avoid leaking internal implementation details.
- Maintain compatibility unless a breaking change has been explicitly approved.
- For the existing FastAPI prototype, follow established route, schema, dependency-injection, and error-handling patterns.

---

12. AI and Reasoning Components

- Separate model-provider integration from core application logic.
- Validate model outputs before passing them to downstream components.
- Handle timeouts, rate limits, malformed responses, and provider failures.
- Do not treat generated text as executable instructions by default.
- Do not allow a model to bypass permission checks or commit policies.
- Preserve the distinction between a generated proposal and an executed action.
- The deterministic reasoning implementation remains the baseline until an AI adapter is explicitly implemented and tested.

---

13. Privacy and Local-First Readiness

- Avoid unnecessary dependence on external services.
- Keep provider and storage integrations replaceable through explicit interfaces.
- Minimize collection and retention of personal data.
- Make persistence behavior explicit and avoid unnecessary transmission of private user information.

14. Documentation and Code Comments

- Documentation must explain important decisions, contracts, setup steps, and non-obvious behavior.
- Comments should explain *why* a decision exists, not merely repeat what the code says.
- Mark provisional designs, assumptions, unresolved questions, and future intentions clearly.

---

15. Formatting and Static Quality

- Follow formatters, linters, type checkers, and test tools already configured in the repository (e.g., PEP 8 for Python).
- Use consistent formatting and imports; avoid unused imports, dead code, and unexplained magic values.

---

16. Performance and Resource Use

- Prefer correct, measurable behavior over premature optimization.
- Avoid unnecessary repeated database, disk, or network operations.
- Use timeouts for external operations and apply pagination or bounded retrieval to potentially large collections.

---

17. Prohibited Practices

Unless explicitly approved and justified, do not rewrite large sections of working code for stylistic reasons, combine independent Kernel responsibilities into a monolithic module, introduce speculative abstractions, disable security checks, ignore failed tests, or claim successful execution without evidence.

---

18. Definition of Done

A code change is complete only when the implementation matches approved task specifications, scope is limited, architectural boundaries are preserved, tests are added/updated and run, documentation is updated, and no unintended secrets or files were introduced.

---

19. Rule Precedence

When requirements conflict, follow this order:

1. Explicit user-approved task requirements and acceptance criteria.
2. Security, privacy, and data-integrity requirements.
3. The approved Truth architecture and project specifications.
4. Existing repository contracts and established conventions.
5. This code standards document.
6. General stylistic preferences.